

# F203 Stage1+2 融合

## CPU 孪生调试经验与 Checklist

ascendc 工程

2026-06-10

### 摘要

本文档记录 `examples/frozen/frozen-exp-mlkem-f203-stage12-encode-matmul-mix/` (原 `incubating` 路径已关闭) 在 CANN 9.0.0、Ascend910B4 CPU 孪生路径上, 从**长时间挂死**到**对拍通过**的完整调试过程。涵盖现象、根因、分步排查、最终方案, 以及可复用的失败/成功经验。真机单趟 MIX 融合待 NPU 实测; CPU 路径采用两趟 launch (encode → 隔离 Cube Matmul)。  
**说明:** 该路线已判决关闭, 本文仅作历史调试实录; 勿从 frozen 抄码。

**勘误:** `exp-sepolyvec8-ntt-k8` 全链 CPU **未跑通** (与 Stage12 同类的 MIX+CrossCore 挂死, 10s 计算预算内无结果)。本文此前误写为「已通过」, 以本文修订为准。

### 目录

|                                                   |          |
|---------------------------------------------------|----------|
| <b>1 目标与基线</b>                                    | <b>2</b> |
| 1.1 融合语义                                          | 2        |
| 1.2 Golden 与隔离基线                                  | 2        |
| <b>2 主要问题 (按发现顺序)</b>                             | <b>2</b> |
| 2.1 问题 1: CPU 对拍长时间无输出                            | 2        |
| 2.2 问题 2: 调试策略——长时间等待                             | 2        |
| 2.3 问题 3: CrossCore 结构与参考不一致                      | 3        |
| 2.4 问题 4: Stage1 tile 参数                          | 3        |
| 2.5 问题 5: CPU 串行仿真 + CrossCore 死锁 (根因之一)          | 3        |
| 2.6 问题 6: MIX 核内 Matmul 在 CPU 上挂死 (根因之二)          | 3        |
| 2.7 问题 7: <code>run.sh</code> 超时与退出码              | 3        |
| 2.8 问题 8: 单独跑内核时库路径                               | 3        |
| <b>3 调试过程 (分步)</b>                                | <b>4</b> |
| 3.1 步骤 1: 确认基线未坏                                  | 4        |
| 3.2 步骤 2: 建立计算预算与 wall-clock 超时                   | 4        |
| 3.3 步骤 3: 对齐 <code>sepolyvec8</code> CrossCore 分块 | 4        |
| 3.4 步骤 4: 分析 <code>cceprint</code>                | 4        |
| 3.5 步骤 5: CPU 两趟 launch (绕过 CrossCore)            | 4        |
| 3.6 步骤 6: 剥离 MIX 宏与 TPipe 试验                      | 4        |

|                                               |          |
|-----------------------------------------------|----------|
| 目录                                            | 2        |
| 3.7 步骤 7: 最终方案——pass2 调用 Stage2 隔离核 . . . . . | 5        |
| <b>4 最终架构</b>                                 | <b>5</b> |
| 4.1 CPU 孪生 (已验证) . . . . .                    | 5        |
| 4.2 NPU / 真机 (设计目标, 待实测) . . . . .            | 5        |
| <b>5 失败经验 (应避免)</b>                           | <b>5</b> |
| <b>6 成功经验 (可复用)</b>                           | <b>6</b> |
| <b>7 后续融合 Checklist</b>                       | <b>6</b> |
| <b>8 当前状态</b>                                 | <b>6</b> |

## 1 目标与基线

### 1.1 融合语义

在单个 MIX 核 (KERNEL\_TYPE\_MIX\_AIC\_1\_2, aicore=1, blockDim=1) 内完成:

- **Stage1 (AIV):**  $se [8,256] \text{ int32} \rightarrow mat\_a [16,256] \text{ int8}$
- **CrossCore:** AIV 写完 `mat_a` 后通知 AIC
- **Stage2 (AIC):**  $mat\_a \times lut [256,512] \text{ int8} \rightarrow mat\_c [16,512] \text{ int32}$

userWorkspace 布局: `mat_a@0` (4096 B) + `mat_c@4096` (32768 B)。

### 1.2 Golden 与隔离基线

Golden 由 `mlkem_ref.encode_mat_a + matmul_int8_i32` 生成; `golden.bin` md5=c5f1ead741f152ab46786 与 Stage2 隔离用例一致。

表 1: 隔离用例基线 (调试前均已通过)

| 用例                                                                         | CPU 耗时  | 说明                                 |
|----------------------------------------------------------------------------|---------|------------------------------------|
| <code>exp-fips203-mlkem-pke-stage1-encode-vec</code>                       | 数秒      | 纯 AIV encode                       |
| <code>examples/frozen/frozen-exp-mlkem-f203-stage2-int8-matmul-cube</code> | 计算 <10s | 纯 Cube Matmul, CPU 已通过             |
| <code>exp-sepolyvec8-ntt-k8</code>                                         | 超时      | Stage1+2+3 全融合, CPU 未通过 (MIX 单趟挂死) |

## 2 主要问题 (按发现顺序)

### 2.1 问题 1: CPU 对拍长时间无输出

**现象:** 编译、`gen_data`、tiling 打印正常; 出现 [TmSim]: Run in serial mode. 后数十分钟无输出; 进程 CPU  $\approx 102\%$ ; 未生成 `output/mat_c_gm.bin`。

**对比:** 同 tiling 的 Stage2 隔离在计算预算内即打印三路 [SUCCESS] [AIC\_0] [AIV\_0] [AIV\_1]。

### 2.2 问题 2: 调试策略——长时间等待

工程约定: 纯计算 (自首次 [TmSim] 或核内实质调度起, 至全部 [SUCCESS] 止) 不应超过 10s。不含软件栈启动、库加载、空转、 workflow 固定开销、编译、`gen_data` 等。超过即异常, 应中止并排查。此前多次长时间 Await, 延误定位。

### 2.3 问题 3: CrossCore 结构与参考不一致

初版将 CrossCoreWaitFlag 与 Stage2MatmulCube 放在同一 ASCEND\_IS\_AIC 块。sepolyvec8 参考为分块: Stage1 块内 AIC 仅 Wait, Stage2 块内再 Matmul。对齐后结构更规范,但仍未解决 CPU 挂死。

### 2.4 问题 4: Stage1 tile 参数

初版沿用 Stage1 隔离:kTileLen=32,kBufferNum=2。融合参考 sepolyvec8 使用 kStage1TileLen=64, kStage1BufferNum=1。非挂死根因,但为减少变量已对齐。

### 2.5 问题 5: CPU 串行仿真 + CrossCore 死锁 (根因之一)

CPU 孪生 [TmSim]: Run in serial mode. 表示串行调度,且默认先跑 AIC。

从 cceprint 可见:

- AIC 入口: wait\_flag\_dev(7) ——启动即等待 Stage1
- AIV 入口: encode 结束后 ffts\_cross\_core\_sync(PIPE\_MTE3, 0x721)

执行顺序变为: AIC 在 WaitFlag 永久阻塞 → AIV 尚未调度 → 死锁。真机 MIX 上 AIC/AIV 并行,不会出现此问题。

### 2.6 问题 6: MIX 核内 Matmul 在 CPU 上挂死 (根因之二)

两趟 launch 绕过 CrossCore 后 (pass1 仅 encode、pass2 仅 matmul), 在融合核内执行 Stage2MatmulCube + IterateAll 仍卡满 10s。

进一步试验:

| 改动                                 | 结果                                        |
|------------------------------------|-------------------------------------------|
| CPU 下去掉<br>KERNEL_TYPE_MIX_AIC_1_2 | pass2 不再无限挂,但出现新错误                        |
| Stage2 内再 new TPipe                | event id cannot be set twice<br>(SIGABRT) |
| 继续共享 TPipe                         | 仍超时; 输出全 0 或半成品                           |

结论: CPU 孪生不宜在 MIX 融合核内直接跑 Matmul IterateAll。

### 2.7 问题 7: run.sh 超时与退出码

初版无 timeout; 曾用子 shell 包裹内核, 超时退出码被吞, 外层误报成功。

### 2.8 问题 8: 单独跑内核时库路径

直接执行 ./ascendc\_kernels\_bbit 可能缺 libpem\_davinci.so; 需完整 tikicpulib / simulator 的 LD\_LIBRARY\_PATH (run.sh 已配置)。

## 3 调试过程（分步）

### 3.1 步骤 1：确认基线未坏

Stage1 / Stage2 隔离均  $\max\_abs\_diff=0 \Rightarrow$  问题在融合胶水层，非 tiling 或 golden。

### 3.2 步骤 2：建立计算预算与 wall-clock 超时

- 计算预算 `KERNEL_COMPUTE_BUDGET_SEC=10`：仅约束核内计算，不含栈启动等固定开销。
- wall-clock `KERNEL_TIMEOUT_SEC`：默认 `STARTUP_MARGIN + COMPUTE_BUDGET`（如  $8 + 10 = 18s$ ），用 `timeout` 包住 `./ascendc_kernels_bbit`，`exit=124` 即判定 hang/deadlock。
- 去掉吞退出码的子 shell。

### 3.3 步骤 3：对齐 sepolyvec8 CrossCore 分块

Wait 移入 `runStage1` 块，`Matmul` 独立 `runStage2` 块；Stage1 tile 改为 64/1。  $\Rightarrow$  仍超时。

### 3.4 步骤 4：分析 cceprint

确认 AIC 先 `wait_flag_dev`、AIV 后 `ffts_cross_core_sync`，结合 serial mode 推断 CrossCore 顺序死锁。

### 3.5 步骤 5：CPU 两趟 launch（绕过 CrossCore）

引入 `g_f203_stage12_cpu_pass`：

- `pass1`：仅 Stage1 encode (`needS1ToS2Sync=false`)
- `pass2`：仅 Stage2 matmul（读 `pass1` 写入的 `mat_a`）

`pass1` 日志示例：

```
[TmSim]: Run in serial mode.
[SUCCESS][AIC_0] exit success!
[TmSim]: Run in serial mode.
[SUCCESS][AIV_0] exit success!
[SUCCESS][AIV_1] exit success!
```

CrossCore 死锁已绕过；`pass2` 在融合核内仍超时。

### 3.6 步骤 6：剥离 MIX 宏与 TPIPE 试验

`#ifndef ASCENDC_CPU_DEBUG` 才设 `KERNEL_TYPE_MIX_AIC_1_2`；禁止 Stage2 内再 new TPIPE。  
`pass2` 仍不可靠。

### 3.7 步骤 7：最终方案——pass2 调用 Stage2 隔离核

CPU 路径：

```
1 g_f203_stage12_cpu_pass = 1;
2 ICPU_RUN_KF(f203_stage12_encode_matmul_mix_custom, blockDim, ...);
3 ICPU_RUN_KF(mlkem_stage2_matmul_custom, blockDim,
4             userWorkspace, lut, userWorkspace + kMatABytes, workspace, tiling)
5             ;
```

CMakeLists.txt 链接 mlkem\_stage2\_matmul\_custom.cpp。

结果：核内计算在 10s 预算内完成，max\_abs\_diff=0，md5 与 golden 一致。

## 4 最终架构

### 4.1 CPU 孪生（已验证）

1. pass1: f203\_stage12\_...\_custom (pass=1, 仅 AIV encode)
2. pass2: mlkem\_stage2\_matmul\_custom (隔离 Cube, 语义与融合 Stage2 一致)

GM 复用: userWorkspace[0..4095]=mat\_a, userWorkspace[4096..]=mat\_c。

### 4.2 NPU / 真机（设计目标，待实测）

单趟 f203\_stage12\_...\_custom: AIV encode → CrossCoreSetFlag → AIC CrossCoreWait Flag → Matmul; 保留 KERNEL\_TYPE\_MIX\_AIC\_1\_2。

## 5 失败经验（应避免）

1. 勿用隔离耗时外推融合耗时：Stage2 计算快 ≠ Stage12 融合一定快。
2. 勿将 sepolyvec8 全链当作 CPU 已通过基线：该用例 CPU 仍超时；仅作 CrossCore 代码结构参考。
3. CPU 孪生 ≠ 真机并行：serial mode + AIC 先跑时，AIV→AIC 的 CrossCore 单趟必死锁。
4. 勿长时间等待：10s 无 [SUCCESS] / 无输出文件即异常。
5. 勿吞子 shell 退出码：timeout 杀进程后脚本须正确失败。
6. 勿在 CPU MIX 融合核内强行 IterateAll：应复用已验证的隔离 Cube 核做 pass2。
7. 勿多趟 launch 重复 new TPipe：触发 event id cannot be set twice。
8. 勿假设对齐 sepolyvec8 结构即可通过：须以 CPU 实测为准。

## 6 成功经验（可复用）

1. **隔离先行**：Stage1 / Stage2 / golden 各自通过后再加胶水。
2. **Golden 与 Stage2 共用 md5**：便于分阶段对拍。
3. **workspace 布局固定**：mat\_a@0 + mat\_c@4096，与 customspec 一致。
4. **分层对照参考**：CrossCore 分块 → sepolyvec8；MIX 角色 → baremix；Matmul 本体 → Stage2 隔离核。
5. **cceprint 定位 Wait/Set 顺序**：比盲猜死锁快。
6. **CPU 路径显式文档化**：STATUS.md、main.cpp 注释说明两趟原因。
7. **区分计算预算与 wall-clock**：COMPUTE\_BUDGET=10s；TIMEOUT 含启动余量。

## 7 后续融合 Checklist

| # | 检查项                     | 说明                            |
|---|-------------------------|-------------------------------|
| 1 | 隔离 CPU 对拍               | Stage1 / Stage2 / golden 各自通过 |
| 2 | 计算预算 10s                | 纯计算；TIMEOUT 另含栈启动余量           |
| 3 | CPU hang 时查 serial mode | AIC 是否先 WaitFlag              |
| 4 | CPU Matmul hang         | pass2 改用隔离 Cube 核；真机再测单趟      |
| 5 | ASCENDC_CPU_DEBUG 分支    | MIX 宏与 launch 策略分路径           |
| 6 | cceprint                | 确认 Wait/Set 与调度顺序             |
| 7 | 三重确认                    | 输出文件 + md5 + verify_result.py |

## 8 当前状态

| 项目                   | 状态                                  |
|----------------------|-------------------------------------|
| customspec + 实现      | 完成                                  |
| CPU aicore=1 对拍      | <b>通过</b> （计算 <10s, max_abs_diff=0） |
| sepolyvec8 全链 CPU    | <b>未通过</b> （10s 计算预算内挂死/超时）         |
| NPU 单趟融合             | 未测                                  |
| aicore=4 / Stage3 融合 | spec 首版不做                           |

验证命令（历史；路线已 frozen，勿作活跃验收）：

```
cd examples/frozen/frozen-exp-mlkem-f203-stage12-encode-matmul-mix
bash run.sh -r cpu -v Ascend910B4 --aicore 1
```

相关路径：examples/frozen/frozen-exp-mlkem-f203-stage12-encode-matmul-mix/STATUS.md、examples/frozen/frozen-exp-mlkem-f203-stage12-encode-matmul-mix/FROZEN.md。